

TEKNISK DOKUMENTASJON · DOKUMENT 1 AV 2

Sørblikket

Systemarkitektur — Komplette Oversikt

En fullstendig teknisk gjennomgang av Sørblikket — fra åpne data fra Stortinget til hva brukeren ser i nettleseren. Dekker datakilder, XML-parsing, kategoriseringsmotor, alle syv sider, React-arkitektur, localStorage, feilhåndtering og teknologivalg.

PROSJEKT	DOKUMENT	VERSJON	STACK
Sørblikket · Agder	Systemarkitektur Oversikt	1.1 · April 2026	React 19 · TypeScript · Vite · Supabase · Stortinget API

Innholdsfortegnelse

Dette dokumentet gir en komplett teknisk oversikt over Sørblikket — fra åpne datakilder på Stortinget og XML-parsing til alle syv React-sider brukeren ser i nettleseren. Hvert kapittel er skrevet for å kunne leses selvstendig.

Seksjon	Tittel	Innhold
1	Systemet i et nøtteskall	Hva Sørblikket er, hvem det løser et problem for, det viktigste arkitekturprinsippet (ingen server-proxy) og dataflyten i fire steg fra API til bruker
2	Systemarkitektur — 5 lag	Visuelt arkitekturdiagram med grundig forklaring av alle fem lag: datakilder, nettverkslaget, XML-parsing, kategorisering og React-UI. Ansvarsfordeling og kommunikasjonsregler
3	Stortingets API — datakilden	Hva Stortingets åpne API er, sesjon-ID-konseptet i detalj, alle syv endepunkter med fullstendig forklaring av returdata, og Supabase som sekundærkilde
4	XML-parsing forklart fra bunnen	Forskjellen mellom JSON og XML, namespace-problemet illustrert med kodeeksempler, alle hjelpefunksjonene i xml.ts med signatur og detaljert forklaring, og et komplett parsing-eksempel steg for steg
5	Kategoriseringsmotor — Levenshtein	Hva Levenshtein-distanse er med eksempler, scoring-algoritmen i fem steg, pseudokode, alle åtte temaer med nøkkelordlister og typisk komité, og begrensninger ved fuzzy matching
6	Frontend — alle 7 sider i detalj	App.tsx som globalt skall, detaljert gjennomgang av alle syv sider: Hjem,

Seksjon	Tittel	Innhold
		Saker/Statistikk, Stemmegivning, Representanter, Partiprogrammer, Om og Personvern — med dataflytt og UX-valg forklart
7	Automatisk oppdatering og minnehåndtering	setInterval med cleanup-funksjon, konsekvenser av manglende cleanup, AbortController for kansellering av fetch-kall, race condition-scenarier med og uten AbortController
8	Tema, Språk og localStorage	De tre temamodiene (system/mørk/lys), localStorage-nøkler, Brave-kompatibel CSS-basert system-modus, toggle-syklusen, språk-autodeteksjon fra navigator.languages og hva som ikke lagres
9	Statusmodell og partiroller	Partiroller etter valget 2025 med Agderbenkens ni partier, statusmodellen for valgløfter (ubehandlet/innfridd/brutt/delvis), og hvordan rollebevissthet påvirker sakstyper
10	Teknologioversikt	Fullstendig tabell over alle teknologier med versjon og begrunnelse, teknologier som bevisst ikke brukes (Redux, Tailwind, React Query, server-proxy), og infrastrukturvalg

To dokumenter: Dette er dokument 1 av 2. Dokument 2 ([dataflyt-teknisk.html](#)) inneholder steg-for-steg kodegjennomgang, miljøvariabler, feilhåndtering per side, driftsanvisning og lokal utviklingsoppsett.

INTRODUKSJON

1

Systemet i et nøtteskall

Sørblikket er et digitalt journalistikkverktøy som gjør det enkelt å følge med på hva Agderbenkens stortingsrepresentanter faktisk gjør — ikke bare hva de lover.

Hva er problemet Sørblikket løser?

Det er tidkrevende og vanskelig å følge med på Stortingets arbeid. Det finnes riktignok åpne data på data.stortinget.no, men disse er kun tilgjengelige som rådata i XML-format — ikke presentert på en måte som er forståelig for folk flest. For å finne ut hva som ble stemt over forrige uke, måtte du gå til Stortingets nettsted, laste ned XML-filer, parse dem manuelt og tolke resultatene selv. Dette krever teknisk kunnskap de fleste ikke har — og selv for de som har det, er det upraktisk å gjenta prosessen regelmessig.

Sørblikket bygger bro mellom rådata og forståelig innsikt. Appen henter, tolker og presenterer data automatisk — slik at brukeren kan fokusere på det politiske innholdet, ikke den tekniske infrastrukturen.

UTEN SØRBLIKKET

For å finne ut hva som ble stemt over forrige uke, måtte du gå til data.stortinget.no, laste ned XML-filer, parse dem manuelt og tolke resultatene selv. Dette krever teknisk kunnskap de fleste ikke har, og er upraktisk for regelmessig bruk. Sakene mangler temaklassifisering og det finnes ingen søkefunksjon på tvers av sesjoner.

MED SØRBLIKKET

Åpne appen, gå til Stemmegivning, klikk en votering. Du ser umiddelbart hva det ble stemt over, hvem som stemte for, hvem som stemte mot, og hva vedtaket ble. Alt uten teknisk kunnskap. Saker er automatisk kategorisert i temaer og søkbare. Data oppdateres hvert femte minutt uten brukerinteraksjon.

Hvem er appen for?

Sørblikket er designet for fire primærgrupper med ulike behov, men samme grunnleggende ønske: innsikt i hva Agderbenkens representanter faktisk gjør på Stortinget.

- **Journalister** — som følger regionalpolitikk og trenger rask tilgang til faktabasert informasjon for å underbygge saker om politisk aktivitet og løfteoppfyllelse.
- **Innbyggere i Agder** — som ønsker å forstå hva "sine" politikere gjør på Stortinget og om de stemmer i tråd med valgløftene de ble valgt på.
- **Studenter i statsvitenskap eller informasjonssystemer** — som studerer politisk representasjon, regionalisme, parlamentarisk praksis eller systemarkitektur i moderne webapplikasjoner.
- **Regionale organisasjoner** — som næringsforeninger, fagforeninger og interessegrupper som vil ha løpende innsikt i saker av særlig betydning for Agder-regionen.

Det viktigste tekniske prinsippet

Ingen serverproxy. Alle API-kall går direkte fra brukerens nettleser til `data.stortinget.no`. Det er ingen mellomserver som Sørblikket drifter. Dette betyr at appen fungerer så lenge Stortingets API er oppe — uavhengig av driftkostnader for servere. Appen er ren statisk frontend som kan hostes gratis på Netlify, Vercel eller GitHub Pages. Stortingets API er åpent for alle og støtter CORS, noe som gjør direkteoppkobling mulig og forsvarlig.

Nøkkeltall

550+

SAKER PER SESJON

**hvert 5.
min**

AUTO-OPPDATERING

7

SIDER I APPEN

Dataflyten i fire steg

1

Stortinget API

Åpent XML-API på `data.stortinget.no`. Ingen API-nøkkel påkrevd. Oppdateres av Stortingets egne



2

fetch() i nettleseren

Direkte HTTP GET-kall fra React-appen via innebygd `fetch()` API. Ingen server-proxy.



3

XML-parsing

`DOMParser` konverterer XML-streng til et `Document`-objekt. Namespace-bevisste



4

React UI

Kategoriserte TypeScript-objekter lagres i `useState`. React re-renderer kun endrede deler av UI. Brukeren ser

systemer.
Inneholder saker,
voteringer,
representanter og
biografier.

AbortController
håndterer
kansellering.
setInterval trigger
re-fetch hvert 5.
minutt.

hjelpesfunksjoner i
xml.ts ekstraherer
data til TypeScript-
objekter.

oppdaterte data
umiddelbart uten
sideinnlasting.

Agderbenken: Betegnelsen på stortingsrepresentantene valgt fra Agder-fylket. Etter valget 2021 og 2025 er Agder representert med representanter fra Høyre, Arbeiderpartiet, Fremskrittspartiet, Venstre, Kristelig Folkeparti, Senterpartiet, Sosialistisk Venstreparti, Rødt og Miljøpartiet De Grønne. Sørblikket henter alle representanter fra `/dagensrepresentanter` og filtrerer på `fylke=Agder` for å vise kun de relevante personene.

Sørblikket

2 · Systemarkitektur — 5 lag

2

ARKITEKTUR

Systemarkitektur — 5 lag

Sørblikket er bygget i fem tydelig adskilte lag. Hvert lag har et klart ansvar, og de kommuniserer kun gjennom veldefinerte grensesnitt. Data strømmer ovenfra og ned — fra ekstern kilde, gjennom nettverket, via parsing og kategorisering, til React-brukergrensesnittet.

LAG 1 – DATAKILDER (EKSTERNE TJENESTER, IKKE KONTROLLERT AV OSS)

Stortingets åpne API

`data.stortinget.no`
Ingen autentisering påkrevd
Returnerer XML (ikke JSON)
Åpen for alle – gratis
Oppdateres av Stortinget selv

Supabase database

PostgreSQL i skyen
REST API over HTTPS
Kun for valgløfter (/parti)
Anon-nøkkel gir lesetilgang
Row Level Security aktivert

▼ `fetch()` direkte fra nettleser · HTTPS · ingen server-proxy mellom

LAG 2 – NETTVERKSLAGET (FETCH API I NETTLESEREN)

`fetch()` API

Innebygd i alle moderne nettlesere
Sender HTTP GET-forespørsler
Returnerer `Promise<Response>`
Ingen ekstra bibliotek nødvendig
Støtter signal for kansellering

AbortController

Kansellerer pågående kall
Unngår race conditions
Ny controller per brukerhandling
signal sendes med `fetch()`
AbortError ignoreres stille

`setInterval` (5 min)

Auto-refresh hvert 5. minutt
Startes i `useEffect` ved mount
Stoppes i `cleanup`-funksjon
Alltid ferske data uten brukerhandling

▼ `res.text()` → rå XML-streng overføres til parser

LAG 3 – PARSING (SRC/UTILS/XML.TS · DOMPARSER INNEBYGD I NETTLESER)

parseXml(tekst)

DOMParser.parseFromString()
 Returnerer Document-objekt
 Sjekker for parsererror
 Konsolladvarer ved ugyldig XML
 Returnerer uansett for caller

hentElementer(doc, navn)

getElementsByTagNameNS("*", navn)
 Wildcard namespace – finner alt
 Array.from(nodeList) konvertering
 Returnerer tom array ved ikke-funn

hentTekst / hentBarn

Henter tekst fra barn-element
 Trygg mot null-elementer
 Returnerer "" ved manglende data
 Kaller .trim() på alt innhold

▼ CaseItem[] og VoteItem[] – TypeScript-objekter – sendes til kategoriseringsmotor og

React state

LAG 4 – KATEGORISERING (SRC/LIB/CATEGORIZATIONENGINE.TS • LEVENSHTAIN FUZZY MATCHING)

Levenshtein-distance

Måler tekstlikhet 0-1
 0 = ingenting felles
 1 = identiske strenger
 Fuzzy matching uten ekstern API
 Beregnes i nettleseren

Temaer fra temaer.ts

Klima, Helse, Utdanning
 Økonomi, Samferdsel, Justis
 Forsvar, Andre (8 temaer)
 Nøkkelordlister per tema
 Konfigurerbart uten kodeendring

categorizeCases()

Beregner score per tema per sak
 Summerer nøkkelord-scorer
 Vinner-tema merkes på saken
 Brukes i filtre og statistikk

▼ React setState() kalles med kategorisert data → trigger re-render

LAG 5 – REACT UI (SRC/PAGES/ + SRC/COMPONENTS/ • REACT 19 • TYPESCRIPT)

useState / useEffect

useState holder data i minnet
 useEffect styrer henting
 Cleanup ved unmount
 Avhengighetsliste kontrollerer re-kjøring

React Router v7

Klientside-ruting
 Ingen hel sideinnlasting
 URL matcher komponent
 useLocation for navigering-events

7 sider

Hjem • Saker • Votering
 Representanter • Parti
 Om • Personvern
 Navbar + tema/språk i alle sider

▼ HTTPS • nettleser rendrer HTML/CSS/JS • bruker ser ferdig grensesnitt

LAG 6 – BRUKER (INGEN INSTALLASJON, INGEN INNLOGGING)

Alle nettlesere støttes

Chrome · Firefox · Safari · Brave · Edge · Mobil · Desktop · Ingen innlogging
nødvendig · Ingen app-installasjon

Lagenes ansvarsfordeling

Lag	Ansvar	Teknologi	Fil/modul
1 — Datakilder	Leverer rå data. Kontrolleres av Stortinget og Supabase, ikke av oss. Vi er avhengige av at disse er oppe.	HTTP/XML, PostgreSQL REST	data.stortinget.no, Supabase
2 — Henting	Henter data fra nettleseren, håndterer HTTP-feil, kansellerer foreldede kall og trigger periodisk re-henting.	fetch(), AbortController, setInterval	useEffect i sidekomponenter
3 — Parsing	Gjør rå XML-streng om til strukturerte JavaScript/TypeScript-objekter med riktig namespace-håndtering.	DOMParser, getElementsByTagNameNS	src/utils/xml.ts
4 — Kategorisering	Beregner tema for saker ved fuzzy matching mot nøkkelordlister. Ingen ekstern tjeneste — alt skjer i nettleseren.	Levenshtein-distanse, nøkkelordlister	src/lib/categorizationEngine.ts
5 — React UI	Viser data til brukeren, håndterer interaksjon, navigering, tema, språk og offline-tilstand.	React 19, TypeScript, CSS Custom Properties	src/pages/*.tsx, src/components/*.tsx

Lagene er løst koblet. XML-parsing vet ingenting om React. Kategoriseringsmotoren vet ingenting om Supabase. `fetch()`-laget vet ingenting om temaer. Dette er et bevisst designvalg: hvert lag eksponerer kun det minimumet av informasjon som neste lag trenger. Konsekvensen er at du kan bytte ut ett lag — for eksempel bytte XML-parsing med en JSON-parser hvis Stortinget noen gang endrer format — uten å røre resten av systemet.



DATAKILDE

Stortingets API — datakilden

Alt innholdet i appen (unntatt valgløfter) henter fra Stortingets åpne API. Å forstå dette API-et er nøkkelen til å forstå hele appen.

Hva er Stortingets åpne API?

Stortinget gjør store mengder data tilgjengelig gratis på <https://data.stortinget.no/eksport/>. Dataene er åpne — ingen registrering, ingen API-nøkkel, ingen kostnad. Du kan åpne en URL i nettleseren og se dataene direkte. Formatet er XML, ikke JSON. Dette er et eldre format, men det er stabilt og veldokumentert av Stortingets IT-avdeling.

Det finnes ingen offisiell rate limiting publisert for API-et, men appen respekterer tjenesten ved å ikke hente data oftere enn hvert femte minutt, og ved å kansellere foreldede kall med AbortController. Alle kall er enkle HTTP GET — ingen POST, ingen autentisering, ingen sesjonsadministrasjon.

Viktig: XML-namespaces. XML-en fra Stortinget bruker namespaces:

`xmlns="http://data.stortinget.no"`. Dette betyr at vanlige DOM-metoder som `querySelector("sak")` IKKE fungerer — de finner ingenting, men gir heller ingen feilmelding. Appen bruker `getElementsByTagNameNS("*", "sak")` med wildcard `*` som løsning. Se seksjon 4 for fullstendig forklaring.

Sesjon-ID — det mest misforståtte konseptet

Alle API-kall som henter data for en spesifikk stortingsperiode krever en sesjon-ID. En "stortingssesjon" er **ikke** det samme som et kalenderår. Sesjonen løper fra oktober ett år til september neste år — dette er Stortingets arbeidssesjon som samsvarer med det parlamentariske kalenderåret, ikke det sivile kalenderåret.

Sesjon-ID	Starter	Slutter	Merknad
2024-2025	Oktober 2024	September 2025	Siste avsluttede sesjon — budsjettdebatten høst 2024 faller inn her
2025-2026	Oktober 2025	September 2026	Gjeldende sesjon per april 2026 — aktiv nå
2026-2027	Oktober 2026	September 2027	Neste sesjon — ikke startet, vil ikke returnere data ennå

Appen beregner sesjon-ID automatisk i `src/config/sesjon.ts`. Logikken: sjekk om nåværende måned er oktober (indeks 9 i JavaScript, som er 0-indeksert) eller senere. Hvis ja, er startåret dette kalenderåret. Hvis nei, er startåret forrige kalenderår. Eksempel: i april 2026 er `getMonth()` lik 3 (april = indeks 3), som er mindre enn 9, så startåret er 2025, og sesjon-IDen blir `2025-2026`.

```
// src/config/sesjon.ts
function beregnetAktivSesjon(): string {
  const nå = new Date()
  // getMonth() er 0-indeksert: januar=0, oktober=9
  const startÅr = nå.getMonth() ≥ 9
    ? nå.getFullYear() // Oktober-deember: dette år
    : nå.getFullYear() - 1 // Januar-september: forrige år
  return `${startÅr}-${startÅr + 1}`
}

// April 2026 → getMonth()=3, 3 < 9, startÅr=2025 → "2025-2026" ✓
// Oktober 2026 → getMonth()=9, 9 ≥ 9, startÅr=2026 → "2026-2027" ✓
```

Alle endepunkter i detalj

/SAKER?SESJONID=2025-2026 — HOVED-ENDEPUNKTET

Henter alle saker registrert i inneværende stortingssesjon. Dette er "hoved-endepunktet" — appen kaller dette ved oppstart og hvert 5. minutt. Svaret er typisk 500–600 XML-elementer, noe som tar 1–3 sekunder å hente og parse avhengig av nettverkshastighet.

Data per sak:

id	Unik saksnummer (f.eks. "20252026-0045")
tittel	Full tittel på saken
korttittel	Kortversjon av tittelen for listvisning

status	"til behandling", "behandlet", "mottatt", m.fl.
type	"representantforslag", "proposisjon", "melding", m.fl.
dato	ISO 8601-dato for registrering
komite/navn	Navn på ansvarlig stortingskomité

/SAK?SAKID=12345 — DETALJERT ENKELT-SAK

Henter detaljert informasjon om én spesifikk sak. Kalles kun når brukeren klikker på en sak i listen — ikke ved oppstart. Inneholder mer informasjon enn /saker-endepunktet, inkludert selve vedtaksteksten.

Ekstra felter (utover /saker):

innstillingstekst	Komitéens innstilling til Stortinget
vedtakstekst	Hva Stortinget faktisk vedtok (kan være lang)
kortvedtak	Kortforklaring av vedtaket — brukes i detaljpanelet
henvisning	Dokument-referanse (f.eks. "Innst. 123 S (2025–2026)")

/VOTERINGER?SESJONID=2025-2026 — ALLE VOTERINGER I SESJONEN

Henter alle voteringer i sesjonen. En "votering" er en enkelt avstemning — én sak kan ha mange voteringer (én per forslag som ble stemt over). En stortingssak om et statsbudsjett kan ha ti eller flere separate voteringer. Brukes på Stemmegivning-siden for å fylle sidebaren.

voteringId	Unik ID for denne voteringen
sakId	Kobling tilbake til saken voteringen tilhører
antallFor	Antall representanter som stemte for
antallMot	Antall representanter som stemte mot
antallIkkeTilStede	Antall fraværende under voteringen
vedtakstekst	Kort beskrivelse av hva det ble stemt over
personligVotering	true/false — false betyr at individuelle stemmer ikke er tilgjengelige

/VOTERINGSRESULTAT?VOTERINGID=X — HVEM STEMTE HVA

Henter hvem som stemte hva i en spesifikk votering. Returnerer en liste over alle representanter som deltok, med stemme per person. Brukes for å vise parti- og representantfordeling i detaljpanelet på Stemmegivning-siden.

representantId	Unik person-ID — kobler til /dagensrepresentanter
navn	Fullt navn på representanten
parti	Partiforkortelse (H, Ap, FrP, SV, Sp, osv.)
stemme	"STEMTE_FOR", "STEMTE_MOT" eller "IKKE_TILSTEDE"

/VOTERINGSFORSLAG?VOTERINGID=X — SELVE FORSLAGSTEKSTEN

Henter selve forslagsteksten som det ble stemt over. Kan inneholde lovtekst, bevilgninger i kroner, politiske vedtakstekster eller rene ordforslag. Viser i detaljpanelet under selve stemmeoversikten. Teksten kan være svært lang for komplekse lovsaker.

/DAGENSREPRESENTANTER — ALLE NÅVÆRENDE REPRESENTANTER

Henter alle nåværende stortingsrepresentanter — det vil si de som sitter i inneværende storting. Appen filtrerer på `fylke = "Agder"` for å vise kun Agderbenken. Returnerer personId, navn, parti, fylke, og URL til representantens offisielle profilbilde fra Stortingets bildarkiv.

/KODETBIOGRAFI?PERSONID=P123456 — BIOGRAFI FOR ÉN PERSON

Henter biografi for én representant — inkludert utdanning, yrkeserfaring og politiske verv. Kalles kun ved klikk på et representantkort — ikke ved sidelast. PersonId-en er en streng som starter med "P" etterfulgt av sifre, f.eks. "P003456".

Supabase — kun for Partiprogrammer-siden

Supabase brukes som database for ett enkelt formål: å lagre og hente valgløfter på `/parti`-siden. Ingenting annet i Sørblikket bruker Supabase. Alle andre seks sider henter eksklusivt fra Stortingets API.

DATABASESTRUKTUR: VALGLØFTE-TABELLEN

lofte_id	UUID PRIMARY KEY
parti	TEXT (Ap, H, FrP, SV, Sp...)
tekst	TEXT (løftets fulle ordlyd)

TILGANGSNØKLER OG SIKKERHET

VITE_SUPABASE_ANON_KEY — brukes i frontend-koden. Gir kun lesetilgang, begrenset av Row Level Security (RLS) i Supabase. Trygg å eksponere i nettleseren — den kan ikke skrive data.

kategori TEXT (Klima, Helse,
Økonomi...)
opprettet TIMESTAMP WITH TIME ZONE

SUPABASE_SERVICE_KEY — brukes kun
av backend-skript for å skrive løfter til
databasen. Aldri til frontend. Oppbevares
kun på server eller i CI/CD-pipeline.

Klienten initialiseres i `src/lib/supabase.ts`. Hvis miljøvariablene `VITE_SUPABASE_URL` eller `VITE_SUPABASE_ANON_KEY` mangler — typisk i et lokalt utviklingsmiljø uten `.env`-fil — returnerer modulen `null` i stedet for å krasje appen ved oppstart. Alle komponenter som bruker supabase-klienten sjekker for `null` og viser en forklarende melding til brukeren om at tjenesten ikke er konfigurert.

4

PARSING

XML-parsing forklart fra bunnen

Stortingets API returnerer XML, ikke JSON. Dette er uvanlig i moderne webutvikling og krever forståelse av hvordan XML-parsing fungerer i nettleseren — og spesielt av namespace-problematikken.

Hva er XML og hvorfor er det annerledes enn JSON?

JSON ser slik ut: `{"id": "12345", "tittel": "Forslag om..."}`. XML ser slik ut: `<sak>
<id>12345</id><tittel>Forslag om...</tittel></sak>`. Begge representerer same data, men XML er mer verbose, hierarkisk og krever en XML-spesifikk parser — du kan ikke bruke `res.json()`. Heldigvis har alle moderne nettlesere en innebygd XML-parser kalt `DOMParser`, slik at vi ikke trenger et ekstra bibliotek.

Den ekstra utfordringen med Stortingets XML er **namespaces**. XML-rotelementet ser slik ut: `<saker xmlns="http://data.stortinget.no">`. Attributtet `xmlns` betyr at alle elementer i dette dokumentet tilhører navnerommet `http://data.stortinget.no`. Dette påvirker på en ikke-åpenbar måte hvordan du kan søke etter elementer i dokumentet.

```
←!— Forkortet eksempel – det faktiske svaret har ~550 <sak>-elementer —→  
<saker xmlns="http://data.stortinget.no">  
  <sak>  
    <id>20252026-0045</id>  
    <tittel>Representantforslag om styrking av kollektivtransport i  
distriktene</tittel>  
    <korttittel>Kollektivtransport i distriktene</korttittel>  
    <status>til behandling</status>  
    <type>representantforslag</type>  
    <dato>2025-11-15</dato>  
    <komite><navn>Transport- og kommunikasjonskomiteen</navn></komite>  
  </sak>  
  ... ~549 saker til ...  
</saker>
```


Namespace-problemet illustrert med kode

Attributtet `xmlns="http://data.stortinget.no"` på rotelementet betyr at alle elementer — `<sak>`, `<id>`, `<tittel>` — implisitt tilhører dette navnerommet. Nettleseren vil da ikke finne dem med søkemetoder som ikke er namespace-bevisste:

VIRKER IKKE — NAMESPACE IGNORERES (STILLE FEIL!)

```
doc.querySelector("sak")
// → null – finner INGENTING

doc.getElementsByTagName("sak")
// → tom HTMLCollection (length=0)

// Ingen feilmelding – koden kjøres normalt
// men returnerer alltid tomt resultat
```

VIRKER — WILDCARD NAMESPACE (*)

```
doc.getElementsByTagNameNS("*", "sak")
// → NodeList med ALLE <sak>-elementer

// "*" = matcher elementer i ALLE namespaces
// Konverteres til array:
Array.from(nodeList)
```

Hjelpesfunksjonene i src/utils/xml.ts forklart

parseXml(tekst: string): Document

Tar en XML-streng og returnerer et DOM-dokument. Bruker nettleserens innebygde `DOMParser`. Sjekker om svaret inneholder et `<parsererror>`-element — dette er DOMParsers måte å si "ugyldig XML" på, siden den aldri kaster en exception, men alltid returnerer et dokument.

```
export function parseXml(xmlTekst: string): Document {
  const parser = new DOMParser()
  const doc = parser.parseFromString(xmlTekst, "application/xml")

  // DOMParser krasjer ikke ved ugyldig XML – den returnerer
  // et dokument med et <parsererror>-element i stedet.
  const feilNode = doc.querySelector("parsererror")
  if (feilNode) {
    console.warn("[xml.ts] Ugyldig XML mottatt:", feilNode.textContent)
  }

  return doc // Returner uansett – la kalleren håndtere tomt resultat
}
```

hentElementer(forelder, tagnavn): Element[]

Finner alle elementer med et bestemt navn, uavhengig av namespace. Returnerer et vanlig JavaScript-array (ikke NodeList, som mangler array-metodene `.map()` og `.filter()`). Returnerer alltid en array — aldri null.

hentBarn(forelder, tagnavn): Element | null

Finner første direkte barn-element med et bestemt navn. Viktig: sjekker `.localName` (uten namespace-prefiks) i stedet for `.tagName` — fordi `.tagName` kan inneholde namespace-prefiks som `"ns0:tittel"` , mens `.localName` alltid er bare `"tittel"` . Returnerer `null` hvis ikke funnet — dette er trygt å videresendes til `hentTekst()` .

hentTekst(forelder, tagnavn): string

Den mest brukte funksjonen i hele kodebasen. Finner et barn-element og returnerer teksten i det. Returnerer aldri `null` eller `undefined` — bruker `?? ""` (nullish coalescing) til å gi tom streng som fallback. Kaller `.trim()` for å fjerne linjeskift og mellomrom som XML ofte inneholder rundt tekstverdier.

Komplett parsing-eksempel: fra HTTP til TypeScript-objekt



Funksjon	Signatur	Beskrivelse
<code>parseXml</code>	<code>(tekst: string) → Document</code>	Oppretter DOMParser, kaller parseFromString, advarer ved parsererror. Returnerer alltid Document.
<code>hentElementer</code>	<code>(doc: Document, navn: string) →</code>	<code>getElementsByTagNameNS("*", navn) + Array.from()</code> . Returnerer tom array hvis ingen

Funksjon	Signatur	Beskrivelse
	<code>Element[]</code>	treff.
<code>hentBarn</code>	<code>(el: Element null, navn: string) → Element null</code>	Søker direkte barn med <code>.localName</code> -matching. Trygg mot null-input.
<code>hentTekst</code>	<code>(el: Element null, navn: string) → string</code>	Returnerer <code>.textContent?.trim()</code> for barn-element. Aldri null — alltid streng (evt. tom).

Stille feil — vær obs! Stortingets XML-navnerom gjør at all parsing som *ikke* bruker

`getElementsByTagNameNS` vil returnere tomme resultater uten noen feilmelding. Appen ser ut til å virke, men lister og saker vises aldri. Dette er en av de mest typiske feilene for nye utviklere som jobber med denne kodebasen.

5

ALGORITME

Kategoriseringsmotor — Levenshtein fuzzy matching

Saker fra Stortingets API har ingen temaklassifisering — bare tittel, type og komiténavn. Kategoriseringsmotoren løser dette ved å beregne tekstlikhet mellom sakstittel og nøkkelordlister med Levenshtein-distance.

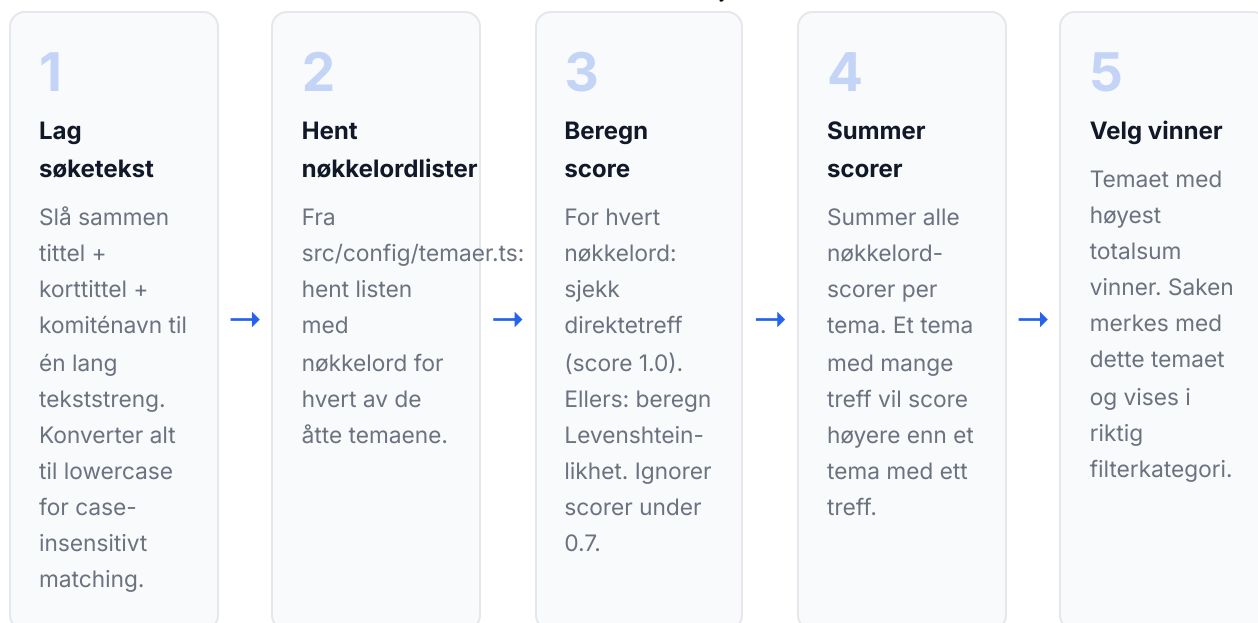
Hva er Levenshtein-distance?

Levenshtein-distance er et klassisk mål fra informatikken som teller **minimum antall enkeltoperasjoner** som kreves for å gjøre én streng lik en annen. De tre tillatte operasjonene er: sette inn ett tegn, slette ett tegn, eller erstatte ett tegn med et annet. Jo lavere distance, jo likere er strengene.

Sørblikket bruker en normalisert variant kalt Levenshtein-likhet: `1 - (distance / max(len1, len2))`. Dette gir et tall mellom 0 og 1, der 1.0 betyr identiske strenger og 0.0 betyr at strengene ikke ligner i det hele tatt.

Streng 1 (nøkkelord)	Streng 2 (i sakstittel)	Levenshtein- distance	Likhet (0–1)	Tolkning
"klima"	"klima"	0	1.00	Identiske — fullt treff
"klima"	"klimat"	1	0.83	Svært likt — én bokstav til
"klima"	"klimagass"	4	0.56	Middels likt — felles stamme
"klima"	"barnehage"	8	0.11	Svært ulikt — ingen kobling

Scoring-algoritmen steg for steg



```

// Pseudokode – illustrerer logikken i categorizationEngine.ts
function categorizeCase(sak: CaseItem): ThemeKey {
  const søketekst = [sak.title, sak.shortTitle, sak.committee]
    .join(" ").toLowerCase()

  let høyesteScore = 0
  let vinnerTema: ThemeKey = "andre"

  for (const [tema, konfig] of Object.entries(TEMA_KONFIG)) {
    let temaScore = 0

    for (const nøkkelord of konfig.keywords) {
      if (søketekst.includes(nøkkelord)) {
        temaScore += 1.0 // Full score ved direkte treff
      } else {
        const score = levenshteinLikhet(søketekst, nøkkelord)
        if (score > 0.7) temaScore += score // Ignorer lave scores
      }
    }

    if (temaScore > høyesteScore) {
      høyesteScore = temaScore
      vinnerTema = tema as ThemeKey
    }
  }

  return vinnerTema
}

```

Temaene, nøkkelordene og typisk komité

Tema	Eksempel-nøkkelord	Typisk komité
Klima	klimautslipp, fornybar, miljø, naturvern, CO2, havvind, biologisk mangfold, karbonfangst	Energi- og miljøkomiteen
Helse	sykehus, legemiddel, psykiatri, pasient, fastlege, omsorg, eldreomsorg, folkehelse, rus	Helse- og omsorgskomiteen
Utdanning	skole, barnehage, lærere, universitet, studiestøtte, SFO, høyskole, videregående	Utdanning- og forskningskomiteen
Økonomi	statsbudsjett, skatt, næringsliv, inflasjon, pensjon, rente, avgift, bevilgning	Finanskomiteen
Samferdsel	vei, jernbane, kollektivtransport, ferge, flyplass, sykkel, bompeng, NTP	Transport- og kommunikasjonskomiteen
Justis	politi, domstol, kriminalomsorg, rettssikkerhet, immigrasjon, asyl, straff	Justiskomiteen
Forsvar	NATO, militær, beredskap, forsvar, Ukraina, sikkerhetspolitikk, FFI, etterretning	Utenriks- og forsvarskomiteen
Andre	Saker som ikke scorer høyt nok for noe annet tema. Standardkategori — sikkerhetsnett for alle ikke-klassifiserte saker.	

Nøkkelordlistene er definert i [src/config/temaer.ts](#) og kan enkelt utvides uten kodeendringer i selve motoren. Legg til et nytt nøkkelord i listen og det vil umiddelbart påvirke kategoriseringen av nye og eksisterende saker ved neste oppdatering.

Begrensning ved automatisk kategorisering: En sak om "klimafondet" (der økonomi og klima begge er relevante) kan plasseres i feil tema avhengig av hvilke nøkkelord som gir høyest samlet score. Kategoriseringen er et praktisk verktøy for filtrering — ikke en autoritativ akademisk klassifisering. Nøkkelordene i [temaer.ts](#) kan justeres løpende for å forbedre treffsikkerheten uten å endre selve algorithmekoden.

6 TIDSSTYRING

Sesjon-ID og tidsstyring

Stortinget opererer ikke i kalenderår, men i "sesjoner" som starter i oktober og slutter i september. Sørblikket beregner alltid riktig sesjon-ID automatisk.

Stortingets sesjonssystem

En stortingssesjon løper fra oktober ett kalenderår til og med september det neste. Sesjon-IDen er alltid i formatet `ÅÅÅÅ-ÅÅÅÅ`, der første årstall er startåret og andre årstall er sluttåret. For eksempel:

Sesjon-ID	Periode	Merknad
<code>2024-2025</code>	Oktober 2024 — September 2025	Siste avsluttede sesjon
<code>2025-2026</code>	Oktober 2025 — September 2026	Aktiv sesjon per april 2026
<code>2026-2027</code>	Oktober 2026 — September 2027	Neste sesjon (ikke startet)

Vanlig misforståelse: Sesjon "2025-2026" er IKKE hele kalenderåret 2025-2026. Den starter i oktober 2025 og slutter i september 2026. I april 2026 er vi midt i sesjonen 2025-2026 — startåret er altså forrige kalenderår.

Automatisk beregning i `src/config/sesjon.ts`

Koden beregner korrekt sesjon-ID basert på dagens dato. JavaScript-måneder er 0-indeksert (januar = 0, oktober = 9). Logikken er:

```
// src/config/sesjon.ts
function beregnetAktivSesjon(): string {
  // Kan overstyres for testing:
  if (TVUNGEN_SESJON) return TVUNGEN_SESJON

  const nå = new Date()
  const år = nå.getFullYear()
```

```
const måned = nå.getMonth() // 0 = jan, 9 = okt

// Oktober (9) eller senere → ny sesjon har startet
const startÅr = måned ≥ 9 ? år : år - 1

return `${startÅr}-${startÅr + 1}`
}
```

Eksempler på beregningen

Dato (eksempel)	getMonth()	Betingelse	Sesjon-ID returnert
15. april 2026	3 (= april)	$3 < 9 \rightarrow$ bruk forrige år	2025-2026
1. oktober 2026	9 (= oktober)	$9 \geq 9 \rightarrow$ ny sesjon startet	2026-2027
31. desember 2026	11 (= desember)	$11 \geq 9 \rightarrow$ ny sesjon startet	2026-2027
1. januar 2027	0 (= januar)	$0 < 9 \rightarrow$ bruk forrige år	2026-2027

Konstanten `TVUNGEN_SESJON` øverst i `sesjon.ts` lar utviklere overstyre den automatiske beregningen ved å sette en hardkodet sesjon-ID. Dette er nyttig for testing med historiske data. Sett den til `null` for å bruke automatisk beregning i produksjon.

7 REACT-APP Frontend — React-appens sider

Sørblikket har syv sider bygget med React 19 og TypeScript, rutet med React Router. Her er en grundig gjennomgang av hva hver side gjør og hvordan den fungerer.

Forside (/) — Hjem.tsx

Forsiden er en statisk side uten API-kall. Den rendres øyeblikkelig og viser en hero-seksjon med kort forklaring av Sørblikket, etterfulgt av fire feature-kort med lenker til de andre sidene. Formålet er å gi nye brukere en rask forståelse av hva systemet gjør og guide dem videre. Ingen tilstand håndteres her.

Saker og statistikk (/statistikk) — Statistikk.tsx + StatistikkDashboard.tsx

Denne siden er delt i to komponenter. `Statistikk.tsx` er en wrapper-komponent som viser en hero-banner med live tall — totalt antall saker og antall behandlet — hentet fra API.

`StatistikkDashboard.tsx` inneholder alt det interaktive innholdet og er delt i to klart definerte seksjoner:

SAKER-SEKSJONEN (ØVERST)

Søkefelt for fritekst, temafiltre som knapper (Klima, Helse osv.), og en saksliste som filtreres live mens brukeren skriver. Klikk på en sak viser et detaljpanel til høyre med sammendrag, vedtakstekst og en lenke til stortinget.no for å lese mer. Alle data hentes fra `/saker` og kategoriseres lokalt i nettleseren.

STATISTIKK-SEKSJONEN (NEDENFOR)

KPI-kort med nøkkeltall, en sesjonsfremdriftsbar som viser hvor langt inn i sesjonen vi er, en temafordelingsvisning og en månedsgraf over saksmengde over tid. Alle tall beregnes fra den samme listen med saker som allerede er hentet til saker-seksjonen — ingen ekstra API-kall.

Stemmegivning (/votering) — Voting.tsx

Denne siden er den mest API-intensive i appen. Den er bygget med en to-kolonne-layout: en venstre sidebar med alle voteringer i sesjonen, og et høyre panel som viser detaljene for den valgte voteringen.

Når brukeren velger en votering fra sidebaren, starter siden **fire parallelle API-kall** via `Promise.all()`:

- `/saker?saksnummer=X` — for sakens tittel og kontekst
- `/voteringsresultat?saksnummer=X` — for alle voteringer i saken
- `/voteringsresultat?voteringid=Y` — for hvem som stemte hva
- `/voteringsforslag?voteringid=Y` — for forslagsteksten

Parallellkjøring er vesentlig her: de fire kallene tar samlet rundt 1–2 sekunder, men i serie ville de tatt 3–6 sekunder. Detaljpanelet viser: vedtaksresultat (vedtatt/ikke vedtatt med farge), antall for/mot/fraværende som fremdriftsbar, og en liste over representantene med parti og stemme.

Representanter (/representanter) — Representanter.tsx

Henter `/dagensrepresentanter` og filtrerer på `fylke=Agder`. Viser et responsivt kort-grid der hvert kort inneholder bilde, navn og parti. Klikk på et kort henter `/kodetbiografi` for personen og viser utdanning, yrkeserfaring og politiske verv i et ekspandert panel. AbortController sørger for at et pågående biografi-kall kanselleres hvis brukeren klikker en annen representant før svaret er kommet.

Partiprogrammer (/parti) — Parti.tsx

Henter valgløfter fra Supabase og er den eneste siden som er avhengig av Supabase-konfigurasjon. Øverst er en parti-velger. Nedenfor er en AI-chatbot der brukeren kan stille spørsmål om et partis valgløfter. Flyten er: brukerens spørsmål sendes til Supabase, som bruker vektorsøk (eller full-text search) for å finne de mest relevante løftene, og returnerer dem til chatboten for visning.

Forslag-chips (forhåndsdefinerte spørsmål) vises som klikkbare knapper og har en fade-effekt på høyre kant som indikerer at det er flere forslag å scrolle til. Chatboten er designet slik at brukerens siste melding scrolles til *toppen* av chat-boksen (ikke bunnen) — se seksjon E i dokument 2 for teknisk forklaring. Hvis Supabase-variabler mangler, viser siden en forklarende feilmelding og chatboten deaktiveres gracefully.

Om Sørblikket (/om) — Om.tsx

Statisk informasjonsside om prosjektet, bacheloroppgavekonteksten ved Universitetet i Agder, partnere og metodikk. Ingen API-kall. Oppdateres manuelt ved behov.

Personvern (/personvern) — Personvern.tsx

Personvernerklæring som forklarer hvilke data Sørblikket samler inn og lagrer. Det korteste svaret: svært lite. Sørblikket lagrer kun to verdier i `localStorage` — brukerens valg av tema og språk. Det settes ingen informasjonskapsler, og ingen persondata sendes til noen server.

Sørblikket

8 · Automatisk oppdatering og minnehåndtering

8

MINNEHÅNTERING

Automatisk oppdatering og minnehåndtering

Alle sider med live-data oppdateres automatisk hvert 5. minutt. Riktig opprydding er avgjørende for å unngå minnelekkasjer og parallelle API-kall.

Auto-refresh med `setInterval`

Hver sidekomponent som henter data fra Stortingets API bruker dette mønsteret i sin `useEffect`:

```
// Kjøres én gang ved mount (tom avhengighetsliste)
useEffect(() => {
  const controller = new AbortController()

  hentData() // Kjør umiddelbart ved mount

  const intervall = setInterval(hentData, 5 * 60 * 1000)

  // KRITISK: cleanup-funksjon
  return () => {
    clearInterval(intervall) // Stopp intervallet
    controller.abort() // Kanseller pågående fetch
  }
}, [])
```

Hva skjer uten cleanup?

Uten `clearInterval` i cleanup-funksjonen vil React starte et nytt intervall hver gang komponenten mountes, men det gamle intervallet vil fortsette å kjøre. Hvis brukeren navigerer frem og tilbake mellom sider tre ganger, vil det kjøre tre parallelle intervaller — alle henter data uavhengig av hverandre. Etter ti navigasjoner vil det være ti samtidige intervaller. Dette er en klassisk minnelekkasje i React-apper.

UTEN CLEANUP — FEIL

Bruker navigerer til /statistikk → 1 intervall starter.

MED CLEANUP — KORREKT

Bruker navigerer til /statistikk → 1 intervall starter.

Navigerer bort → intervallet fortsetter.
Navigerer tilbake → 2. intervall starter.
Navigerer bort og tilbake igjen → 3 intervaller kjører.

Resultat: Minnelekkasje og unødvendig nettverkstrafikk.

Navigerer bort → React kaller cleanup → intervallet stoppes.

Navigerer tilbake → nytt intervall starter (kun 1 aktiv).

Resultat: Alltid nøyaktig ett intervall per aktiv side.

AbortController — kansellering av pågående kall

I tillegg til intervall-cleanup brukes `AbortController` i `Votering.tsx` og `Representanter.tsx` for å kansellere pågående fetch-kall. Dette er nødvendig fordi:

- Hvis brukeren klikker raskt mellom to voteringer, vil begge API-kallene pågå parallelt. Svaret som kommer sist vil overskrive det korrekte resultatet.
- `AbortController` sikrer at det første (nå utdaterte) kallet kanselleres før det andre sendes.
- En kansellert `fetch()` kaster en `AbortError`. Denne feilen ignoreres stille — den er forventet og skal ikke vises som feilmelding til brukeren.

Huskeregel: Alle `setInterval`-kall trenger en tilhørende `clearInterval` i cleanup. Alle `AbortController`-instanser skal ha `abort()` kalt i cleanup. Brudd på denne regelen fører til hard-to-debug minnelekkasjer.

BRUKERPREFERANSER

9

Tema, Språk og localStorage

Sørblikket husker brukerens preferanser mellom besøk via `localStorage`. Mørk modus er implementert på en måte som fungerer i alle nettlesere, inkludert Brave.

Temalagring

Temavalget lagres i `localStorage` under nøkkelen `"sørblikket-tema"`. Det finnes tre mulige tilstander:

Verdi i localStorage	DOM-tilstand	Mekanisme
<code>null</code> (ikke satt)	<code><html></code> uten <code>data-theme</code>	CSS <code>@media (prefers-color-scheme: dark)</code> tar over. Følger operativsystemets innstilling.
<code>"dark"</code>	<code><html data-theme="dark"></code>	CSS <code>[data-theme="dark"]</code> -selektor overstyrer media-query. Alltid mørkt uavhengig av OS.
<code>"light"</code>	<code><html data-theme="light"></code>	CSS <code>[data-theme="light"]</code> -selektor låser lyst tema. Alltid lyst uavhengig av OS.

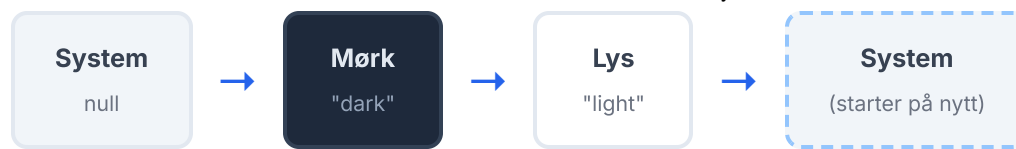
Brave-kompatibel system-modus

I system-modus (`null`) er CSS `@media (prefers-color-scheme: dark)` den primære mekanismen. Dette er et bevisst valg fordi Brave-nettleseren ikke alltid sender `change`-hendelser til JavaScript's `matchMedia`-API ved temabytte i operativsystemet. Ved å basere system-modus på CSS i stedet for JavaScript, fungerer mørk modus alltid — selv i Brave — uten JavaScript-events.

JavaScript-hendelsene `visibilitychange` og `focus` brukes som fallback for å synkronisere `data-home-theme`-attributtet slik at React-staten holdes oppdatert når brukeren bytter fane og tilbake.

Toggle-syklusen

Bruker kan sykle gjennom temaene ved å klikke på temaikonet i hamburgermenyen. Rekkefølgen er alltid: System → Mørk → Lys → System → ...



Språklagring og auto-deteksjon

Språkvalget lagres i `localStorage` under nøkkelen `"sørblikket-lang"`. Støttede verdier er `"no"` (norsk) og `"en"` (engelsk).

Hvis det ikke finnes noe lagret språkvalg, detekteres brukerens foretrukne språk automatisk fra nettleseren via `navigator.languages`. Koden sjekker om noen av nettleserens foretrukne språk starter med `nb` (bokmål), `nn` (nynorsk) eller `no` (norsk generelt). Hvis ja, settes norsk som standard. Ellers brukes engelsk.

Hva som IKKE lagres

For fullstendighetens skyld: utover tema og språk lagrer Sørblikket ingenting lokalt. Det settes ingen informasjonskapsler. Det sendes ingen persondata til noen server. Ingen sporingspikslers, ingen analyseverktøy, ingen tredjeparts-skript.

TEKNOLOGI

10 Teknologioversikt

Fullstendig oversikt over alle teknologier brukt i Sørblikket, med versjon og begrunnelse for valget.

Frontend-teknologier

Teknologi	Versjon	Rolle	Begrunnelse
React	19	UI-rammeverk	Industristandard for komponentbasert UI. React 19 gir bedre ytelse og nye hooks. Stor dokumentasjon og støtte.
TypeScript	5.x	Type-sikkerhet	Fanger feil ved kompilering. Essensielt når XML-parsing produserer uforutsigbare datastrukturer. Bedre editor-støtte.
Vite	5.x	Build-tool og dev-server	Ekstremt rask dev-server med HMR (Hot Module Replacement). Mye raskere enn Create React App. Innebygd TypeScript-støtte.
React Router	6.x	Klientside-ruting	Standard for SPA-navigasjon i React. Gir URL-basert navigasjon uten sidelasting. <code>useLocation</code> brukes til å lukke mobilmeny ved navigering.
CSS Custom Properties	—	Styling og temaer	Ingen CSS-rammeverk brukes. Custom properties (variabler) brukes for fargetemaer. Gir full kontroll og minimalt bundles.

Datalag-teknologier

Teknologi	Versjon	Rolle	Begrunnelse
Stortinget Open API	—	Primær datakilde	Offisiell, åpen kilde for stortingsdata. Ingen API-nøkkel, ingen kostnad, alltid oppdatert. XML-format er utfordrende men håndterbart.

Teknologi	Versjon	Rolle	Begrunnelse
Supabase	—	Database for valgløfter	PostgreSQL-basert Backend-as-a-Service. Gratis plan tilstrekkelig. Innebygd autentisering og Row Level Security. Enkelt JavaScript SDK.
DOMParser (nettleter)	Innebygd	XML-parsing	Innebygd i alle moderne nettlesere. Ingen ekstra bibliotek nødvendig. Støtter namespace-bevisst parsing via <code>getElementsByTagNameNS</code> .
fetch API (nettleter)	Innebygd	HTTP-kall	Innebygd i alle moderne nettlesere. Ingen axios eller annet bibliotek nødvendig. Støtter <code>AbortController</code> for kansellering.

Infrastruktur

Teknologi	Rolle	Merknad
Netlify / Vercel	Hosting av statisk frontend	Vite bygger appen til statiske filer. Kan deployes til enhver statisk host. Ingen server-prosess å drifte.
localStorage	Brukerpreferanser	Lagrer tema og språk. Ingen cookies, ingen server-lagring av brukerdatabe.
AbortController	Kansellering av HTTP-kall	Innebygd browser API. Brukes for å unngå race conditions og minnelekkasjer.

Hva som bevisst IKKE brukes

INGEN REDUX / ZUSTAND

Appens tilstandshåndtering er enkel nok til at innebygd React `useState` og `useEffect` er tilstrekkelig. Global state er unødvendig kompleksitet her.

INGEN TAILWIND CSS

Håndskrevet CSS med custom properties gir full kontroll over temaer og mørk modus uten å introdusere et build-avhengig CSS-rammeverk.

INGEN SERVER-PROXY

Stortingets API er åpent og CORS-vennlig — ingen server nødvendig. Appen er 100% statisk og billig å hoste.

INGEN REACT QUERY

Data-hentinglogikken er enkel nok til å implementeres direkte med `useEffect`. React Query ville vært overkill og lagt til unødvendig kompleksitet.

